

ChatHPC: Building the Foundations for a Productive and Trustworthy AI-Assisted HPC Ecosystem

Pedro Valero Lara
Oak Ridge National Laboratory
(ORNL)
Knoxville, USA
valerolarap@ornl.gov

Aaron Young
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
youngar@ornl.gov

Jeffrey S. Vetter
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
vetter@ornl.gov

Zheming Jin
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
jinz@ornl.gov

Swaroop Pophale
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
pophaleess@ornl.gov

Mohammad Alaul Haque Monil
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
monilm@ornl.gov

Keita Teranishi
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
teranishik@ornl.gov

William F. Godoy
Oak Ridge National Laboratory
(ORNL)
Oak Ridge, USA
godoywf@ornl.gov

Abstract

ChatHPC democratizes large language models for the high-performance computing (HPC) community by providing the infrastructure, ecosystem, and knowledge needed to apply modern generative AI technologies to rapidly create specific capabilities for critical HPC components while using relatively modest computational resources. Our divide-and-conquer approach focuses on creating a collection of reliable, highly specialized, and optimized AI assistants for HPC based on the cost-effective and fast Code Llama fine-tuning processes and expert supervision. We target major components of the HPC software stack, including programming models, runtimes, I/O, tooling, and math libraries. Thanks to AI, ChatHPC provides a more productive HPC ecosystem by boosting important tasks related to portability, parallelization, optimization, scalability, and instrumentation, among others. With relatively small datasets (on the order of KB), the AI assistants, which are created in a few minutes by using one node with two NVIDIA H100 GPUs and the ChatHPC library, can create new capabilities with Meta's 7-billion parameter Code Llama base model to produce high-quality software with a level of trustworthiness of up to 90% higher than the 1.8-trillion parameter OpenAI ChatGPT-4o model for critical programming tasks in the HPC software stack.

CCS Concepts

• Computing methodologies → Natural language processing.

Keywords

Large Language Models, Productivity, Trustworthiness, High Performance Computing.

ACM Reference Format:

Pedro Valero Lara, Aaron Young, Jeffrey S. Vetter, Zheming Jin, Swaroop Pophale, Mohammad Alaul Haque Monil, Keita Teranishi, and William F. Godoy. 2025. ChatHPC: Building the Foundations for a Productive and Trustworthy AI-Assisted HPC Ecosystem. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3712285.3759787>

1 Introduction

High-performance computing (HPC) applications demand effective parallelization, optimization, and portability to leverage modern heterogeneous architectures. However, these tasks often require significant expertise and labor, which pose challenges for developers when rapid turnaround is required. Recent advances in large language models (LLMs) offer a promising solution by automating and enhancing complex programming tasks [1–4], thereby accelerating the development process.

In this paper, we introduce ChatHPC: a process and tool chain for developing AI assistants fine-tuned from Code Llama [1] and specifically designed to boost productivity in HPC software development. Our initial ChatHPC models target three critical areas: parallelizing sequential code, optimizing existing parallel implementations, and porting applications across heterogeneous platforms. By tailoring Code Llama—a robust foundation model for code generation—to the nuances of the HPC software stack (e.g., programming systems, math libraries, I/O, tools), we enable developers to generate accurate, high-performance, and platform-specific code while requiring significantly fewer manual tasks. Our approach leverages domain-specific fine-tuning and uses datasets that include HPC benchmarks



This work is licensed under a Creative Commons Attribution 4.0 International License. SC '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1466-5/25/11

<https://doi.org/10.1145/3712285.3759787>

and real-world applications to ensure practical utility. These efforts aim to create a more cost-effective and productive AI-assisted HPC software ecosystem.

1.1 Key Contributions

Specifically, this work offers the following:

- (1) A lightweight software infrastructure (i.e., the ChatHPC library) that enables HPC experts to efficiently create and supervise trustworthy AI capabilities for critical HPC software tasks, thereby lowering the barrier to AI integration in HPC systems while maintaining reasonable computational requirements.
- (2) A demonstration and evaluation of ChatHPC on important components of the HPC software stack, including programming systems (e.g., Kokkos [5], IRIS [6], SYCL, OpenMP, CUDA), scientific libraries (e.g., MAGMA [7]), tools (e.g., TAU [8]), and I/O (e.g., ADIOS2 [9]). We also show the challenges for the unique patterns and characteristics of each product.

Ultimately, this work demonstrates how specialized LLMs can bridge the gap between general-purpose code generation and the stringent, unique requirements of HPC software to pave the way for more accessible and efficient scientific computing.

2 Background

2.1 LLMs

LLMs [10] are trained on large datasets, including public software repositories that contain HPC codes, and the larger LLMs can consist of billions or even trillions of parameters. Training involves optimizing the LLM's parameters by predicting the next token (or word segment) and updating the weights (parameters) depending on the difference between the actual next word and the predicted next word. The way to interact with an LLM is to use text as the input. This text, known as a *prompt*, is then tokenized or split into chunks of characters (UTF-8 encoding). LLMs create embeddings as a representation of the meaning of the tokens by using a high-dimensional vector space. After the embeddings are created, the model uses self-attention and other mechanisms to generate the output. The ordering is captured by using positional encodings. The LLMs then use the set of embeddings to decode the embedding into the next most likely token. The iterative generation of the next most likely token is used to produce the response tokens. This output is then converted back into text via de-tokenization to generate the final output. The process of predicting the next word or token is known as *causal language modeling* [11] and enables LLMs to carry out specialized tasks, including code generation.

2.2 Code Llama

Using LLMs for programming is a very active and evolving area of research within the AI community. LLMs made for this purpose include CodeGeeX [12], StarCoder [13], Code Llama [14], CodeRosetta [15], and DeepSeek-coder [16]. Most of these models provide very similar capabilities depending on the parameters

used [16]. However, other factors, such as accessibility, documentation, community, support for modern hardware/software, interoperability, and compatibility with other models or HPC components, made Code Llama our first choice. Code Llama is trained for multiple programming languages (Python, C++, Java, PHP, TypeScript, C#, and Bash) to cover a wide range of applications. Code Llama achieves state-of-the-art performance compared to other open models on several code benchmarks with support for large input contexts. Its ability to work with long sequences of code is based on the use of measuring perplexity [17] over 4 million tokens from the code dataset and key retrieval [18]. Code Llama uses execution feedback to select data to train the model—a self-instruction dataset that resulted in about 14,000 question-tests-solution triplets, 62,000 interview-style programming questions, and about 52,000 questions. Code Llama is based on the Llama 2 models, which are trained on 2 trillion tokens of text, including 80 billion tokens of code. Code Llama is the result of tuning Llama 2 models on 500 billion extra tokens that consist mostly of code (85%).

3 Related Work

Unsurprisingly, there is a growing interest in evaluating the impact of LLMs on HPC software [19], and this is becoming a very active area of research. For example, LM4HPC [20] is a language model specifically designed for HPC and demonstrates success in specific tasks such as code similarity analysis, parallelism detection, and OpenMP question answering. Nichols et al. [21] created the trained HPC-Coder model based on the DeepSpeed [22] deep learning model. They found varying degrees of success in code completion for OpenMP pragmas and MPI calls. Another example is LLM4VV [23], which is a fine-tuned model that explores the capabilities of GPT-4 and Llama 2 to generate tests for the compiler functionality of the directives-based OpenACC parallel programming model and shows a high degree of success. Some recent work has explored using LLMs to translate Fortran codes: Lei et al. [24] created a dataset to train LLMs for HPC code translation from Fortran OpenMP to C++ OpenMP, and Zhou et al. [25] proposed a proof-of-concept that uses LLMs to translate single-threaded Fortran to Python/JAX. Godoy et al. [26, 27] and Valero-Lara et al. [28] evaluated current LLM capabilities that included GitHub Copilot and Llama 2 for generating correct and functional representative HPC kernels on modern HPC programming models. More recently, Valero-Lara et al. [29] developed ChatBLAS as the first AI-generated, portable basic vector-vector (BLAS level-1) library, which achieved competitive or better performance than vendor-specific math libraries by using prompt engineering and fine-tuning on top of ChatGPT models.

4 Design and Implementation of ChatHPC

The ChatHPC ecosystem is composed of the ChatHPC library and a collection of fine-tuned AI assistants created by the ChatHPC library for the specific tasks that constitute the essential parts of the HPC software stack. As Figure 1 illustrates, the creation of a new AI assistant in ChatHPC consists of a three-step iterative process: **(1) fine-tuning, (2) testing, and (3) refinement.**

The **(1) fine-tuning** process creates a parameter-efficient adapter for each AI assistant using the *(A) training data* and a

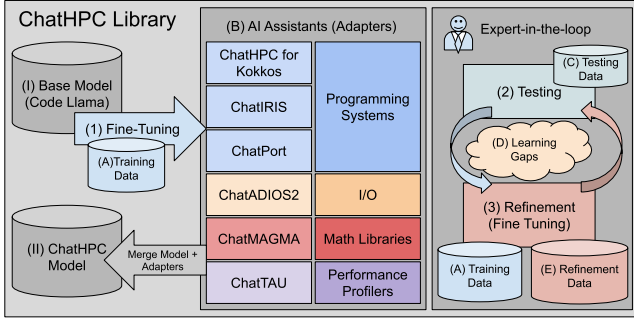


Figure 1: The ChatHPC ecosystem.

JavaScript Object Notation (JSON) format file with a set of prompt-context-answer tuples and data about the capabilities to be created (Figure 2). In our case, this data usually occupies only a few KB of memory. The (B) *adapters* are small and trainable modules added to the base LLM to enable lightweight and efficient fine-tuning for specific tasks, thereby preserving the original model knowledge while adapting to new scenarios. Each adapter (AI assistant) consumes about 100 MB of memory. The (I) Base Model (Code Llama 7 billion [13 GB] in our case) does not need to be replicated. This is a scalable approach that allows the HPC community to work on multiple AI assistants simultaneously with relatively modest resources. (2) **Testing** focuses on checking if the model is built correctly according to its design specifications and ensures that the model actually meets the intended user needs and performs well. This occurs after fine-tuning when a functional AI assistant is available for testing using (C) *testing data* that is different from that used during fine-tuning (training data). The testing data may contain prompts about capabilities not related to the capabilities created (e.g., porting CUDA applications to Kokkos when no data about CUDA was used for fine-tuning). (3) **Refinements** are made after testing when (D) *learning gaps* (i.e., prompts in the testing data that did not receive good responses) are identified. This expert-guided refinement consists of complementing the training data with additional information called (E) *refinement data*, which covers the deficiencies found during validation for a new fine-tuning process. This iterative process allows AI assistants to dynamically learn and adapt to complex tasks over time as more learning gaps are identified. Although a user can interact with each AI assistant (adapters) individually, these adapters not also be merged with the base model's weights, if necessary, to create a single LLM model file—(II) ChatHPC Model in Figure 1.

The ChatHPC library relies on well-known, robust, efficient, and widely used tools and languages, including the aforementioned open-source Meta Code Llama LLM [14], the LoRA (low-rank adaptation) tool used for fine-tuning pretrained LLMs, and the open-source PyTorch [30] Python framework used for building machine learning models. We also implemented a Python command line interface (CLI) library in ChatHPC for ease of use (Figure 3). By integrating these widely used tools and methods, we aim to make the ChatHPC library interoperable and compatible with other important efforts within the AI community (e.g., Ollama [31], Hugging Face [32]) while ensuring usability.

```

1 [
2   {"question": "Can you translate this HIP code to Kokkos?",
3     __global__ void stencil (int n, float dx, float dy, float *u,
4       float *u_new, float *f) {
5       int i = blockIdx.x * blockDim.x + threadIdx.x;
6       int j = blockIdx.y * blockDim.y + threadIdx.y;
7       if (i > 0 && i < n-1 && j > 0 && j < n-1){
8         u_new[i][j] = 0.25 * (u[i-1][j] + u[i][j+1]
9           + u[i][j-1] + u[i+1][j]) + f[i][j];
10      }
11    },
12    "context": "HIP to Kokkos translation",
13    "answer":
14      "Kokkos::parallel_for(\\\"stencil\\\",
15        Kokkos::MDRangePolicy<Kokkos::Rank<2>>
16          ({1, 1}, {n-1, n-1})),
17        KOKKOS_LAMBDA(const int i, const int j) {
18          u_new(i,j) = 0.25 * (u(i-1,j) + u(i,j+1)
19            + u(i,j-1 + u(i+1,j) + f(i,j));
20        });"
21  },
22  ...
23 ]

```

Figure 2: Example of a JSON item used for the ChatHPC for Kokkos AI assistant.

ChatHPC CLI:

```

1 $ chatnpc train # Finetune the assistant.
2 $ chatnpc verify # Verify the assistant on training set.
3 $ chatnpc test # Test the assistant on unseen data.
4 $ chatnpc run # Interactively run the assistant.

```

Interactive run session:

```

1 $ chatnpc ()> /context
2 Context: Introduction to Kokkos
3 $ chatnpc (Introduction to Kokkos)> What is LayoutLeft?
4 LayoutLeft refers column-major layout where consecutive entries in
  the same column of a 2-D array are contiguous in memory.

```

Figure 3: Example of the CLI for the ChatHPC Python library.

We elected to create a handful of AI assistants based on (1) their target component's importance to the HPC community, (2) the need for a sufficiently broad scope to cover different types of functionality (e.g., I/O, parallel programming models, libraries), and (3) our expertise on these topics. As shown, these initial efforts can be easily extended to other HPC components or capabilities with the ChatHPC library.

5 Evaluation of ChatHPC

Our analysis focuses on evaluating the level of trustworthiness for the different AI assistants created by ChatHPC. **We measure this trustworthiness as the percentage of the prompts that were answered correctly by the ChatHPC AI assistants and the reference LLMs (Code Llama base model and ChatGPT's 4o model).** We also created a collection of expert-supervised testing datasets for each AI assistant. Every testing dataset contains a collection of prompts that can be related to documentation or an API of the target tools and relate to some of the other capabilities that involve code transformation (e.g., portability of CUDA codes to OpenMP) or even emergent capabilities not present in the training data. All prompts from the testing datasets and the possible correct answer(s) were carefully constructed by the experts. Additionally, we included performance analysis for some of the codes generated by ChatHPC assistants (ChatHPC for Kokkos and ChatADIOS2) to evaluate speedup and scalability.

Although the methodology for creating AI assistants is the same (Figure 1), the efforts required are different depending on the AI assistant to be created. These efforts may require more or less data depending on the number of capabilities or contexts to be created in a given AI assistant. Also, depending on the similarities between prompts (e.g., input source code) and the potential responses (e.g., output source code), we may need larger or smaller training datasets (e.g., the portability of CUDA codes to OpenMP offloading codes is more demanding than the portability of CUDA codes to SYCL). In general, the more recurring repetitive patterns are found between prompts and responses, the better the LLM can identify the necessary code transformations. In fact, this allows AI assistants to make predictions and understand new situations or capabilities based on the knowledge gained from these repeating patterns and without additional fine-tuning data. Additional details are provided for each AI assistant in the relevant sections below.

For all the results presented in this work, we used one computational node equipped with two NVIDIA H100 GPUs. In addition, the time for the creation of each AI assistant presented in this work was always fewer than 15 minutes. The entire collection of prompt-context-answer tuples used for training, verification, refinement, and testing, as well as the corresponding answers generated for each AI assistant, are easily verifiable and available in the attached appendices. In our analysis, we include the results provided by Meta’s Code Llama base model without fine-tuning (7 billion parameters) and OpenAI’s ChatGPT-4o model (a mixture of 8×222 billion parameter LLMs for a total of approximately 1.8 trillion parameters). This puts ChatGPT-4o at about $257\times$ more parameters than our Code Llama base model.

This evaluation answers the following questions: (1) Is it possible to create trustworthy HPC capabilities by fine-tuning LLMs? (2) How much effort (e.g., data, time) would that require? (3) How well do these capabilities perform when compared with state-of-the-art LLMs?

5.1 ChatHPC for Kokkos

Kokkos [5] is an open-source performance-portability C++ template and metaprogramming library that aims to be architecture agnostic so that programmers can move beyond the low-level details of vendor-specific or open-source programming models and the varying characteristics of the targeted hardware architectures. Kokkos is implemented as a template library on top of some popular HPC programming models (e.g., CUDA, HIP, OpenMP, OpenACC, HPX) [33] and aligns with developments in the C++ standard. Given its utility, Kokkos made our list of targets for an AI assistant, and here we present the aptly named ChatHPC for Kokkos.

Data: We divide the data into different contexts of learning or capabilities that we want to implement in ChatHPC for Kokkos: Documentation, Installation, Development, Parallelization, and Translation. Each context is focused on one specific target. For example, while Documentation focuses on the Kokkos API and targets new Kokkos users, Parallelization and Translation provide assistance for auto-parallelization (e.g., parallelizing sequential kernels to Kokkos) and porting kernels (e.g., porting CUDA kernels to Kokkos), respectively. Our approach consists of *not* providing all the information that we could provide for the initial fine-tuning. For instance, for

Table 1: ChatHPC for Kokkos testing (percentage of prompts with correct responses). [*]No data for CUDA or OpenACC programming models was used for the fine-tuning (e.g., self-learning).

Context	Code Llama	ChatHPC for Kokkos Initial	ChatHPC for Kokkos Refinement	ChatGPT
Documentation	9.50%	89.00%	-	81.00%
Installation	27.20%	78.00%	-	45.50%
Development	0.00%	85.00%	-	15.14%
Parallelization	0.00%	45.50%	90.90%	66.70%
Translation	0.00%	55.58%	85.85%	33.40%
OpenACC*	0.00%	53.20%	87.20%	41.30%
CUDA*	0.00%	58.70%	83.50%	31.20%

code translation, we focused on only HIP and OpenMP models so we can evaluate the capabilities of ChatHPC for Kokkos to do the same for other models (e.g., CUDA or OpenACC). Similarly, we focus on those kernels that are widely used by HPC applications, including Basic Linear Algebra Subprograms (BLAS), stencil-based algorithms, computation on sparse formats, and iterative solvers, among others. Initially, we do not provide a complete list of kernels but rather a subset of representative cases, including level-1 BLAS kernels, 1D stencil operations, or Sparse Matrix Vector Multiplication using a CSR format. For the initial training data, we used about 70 different question-context-answer samples in total.

Analysis: After the initial fine-tuning, we test ChatHPC for Kokkos by using a different set of prompts that were not in the training data used for the initial fine-tuning. These new source codes correspond to other more complex operations, including level-2 and level-3 BLAS operations, more complex (five- and seven-point) stencil applications (e.g., Euler, lattice Boltzmann solvers), computation on sparse matrices using different formats (e.g., CSC, COO), and iterative solvers based on the conjugate gradient algorithm, among others.

Although ChatHPC for Kokkos provides good results in some capabilities and reaches high trustworthiness levels for some contexts of learning, including in Documentation and Installation (see *ChatHPC for Kokkos-initial* in Table 1), we find that some capabilities, including Parallelization and Translation, need to be refined because of learning gaps. Although the initial fine-tuning provides very good results for operations on vectors and matrices, such as many of the BLAS kernels, or some stencil computations, some other sparse formats and iterative algorithms do not provide the expected results. These learning gaps can have different causes, such as insufficient training data during the initial fine-tuning. We can fill the learning gaps by adding specific data into different contexts of learning for the capabilities that our AI assistant cannot process as expected. This requires adding about 12% more data to the training set to create the refinement set, and the new fine-tuning iteration takes about 10% more time than the initial fine-tuning. Notably, no data was added for the CUDA and OpenACC programming models during this new iteration.

After one more fine-tuning iteration (refinement), we proceeded with a new verification pass that showed a notable improvement in ChatHPC for Kokkos’s capabilities (Table 1). By comparing our

newly refined implementation with the Code Llama base implementation (no fine-tuning), we can confirm the addition of Kokkos capabilities in ChatHPC for Kokkos. This newly created AI assistant can now provide assistance for Kokkos programmers and users in five different contexts. ChatHPC for Kokkos also provides better results than the ChatGPT model, even in contexts that are relatively easy to find on the Internet (e.g., Kokkos documentation) [34]. We see more prominent differences in the rest of the contexts, which are more related to specific HPC tasks, such as the translation of codes from different programming models to Kokkos or auto-parallelization. Our analysis includes the testing of non-fine-tuned (self-learned) capabilities, such as the translation (porting) of OpenACC and CUDA applications to Kokkos codes. Notably, no OpenACC or CUDA codes were used for the initial training data or refinement iterations. We benefit from the similarity of the CUDA (and OpenACC) model to the HIP (and OpenMP) model. The prompts used for this case are the same ones used for the translation of OpenMP and HIP codes to Kokkos codes, but we use the OpenACC and CUDA syntax instead.

Translate the next CUDA code to Kokkos

```
1 __global__ void sdot(int n, float *x, float *y, float *result) {
2   int i = threadIdx.x + blockIdx.x * blockDim.x;
3   float update = 0.0;
4   if (i < SIZE) {
5     update = x[i] * y[i];
6     atomicAdd(result, update);
7   }
8 }
```

Chat for Kokkos output:

```
1 Kokkos::parallel_reduce(SIZE,
2   KOKKOS_LAMBDA(int i, float &update){
3     update += x(m) * y(m);
4   },
5   Kokkos::Sum<float>(result));
```

Figure 4: Example of a conversation with ChatKokkos.

As illustrated in Figure 5, ChatHPC for Kokkos can translate vendor-specific (and non-optimized) CUDA code (Figure 4) and provides a Kokkos code that can reach an acceleration of 300× over the input code and compete with NVIDIA’s cuBLAS performance on the same platform (one NVIDIA H100 GPU, nvc++ 24.9-0, and Kokkos 3.0). Unlike CUDA, Kokkos provides a portable and easy-to-use API that carries out the target-specific optimizations in each of the back ends. ChatHPC for Kokkos benefits from the capabilities developed in Kokkos to provide portable, optimized, and high-performance codes from vendor-specific applications, thereby enhancing productivity in the development of codes for HPC and scientific computing.

Another important capability of ChatHPC for Kokkos is the auto-parallelization of kernels (Figure 6)—a process that computes an AI convolution and results in a better performing code implemented in Kokkos, with an acceleration of about 4×10^5 versus the sequential code used as the input (prompt).

5.2 ChatADIOS2

HPC parallel I/O is a relevant issue for several large data producer/consumer applications that run at scale. Earth science, fusion, computational fluid dynamics, and materials science applications

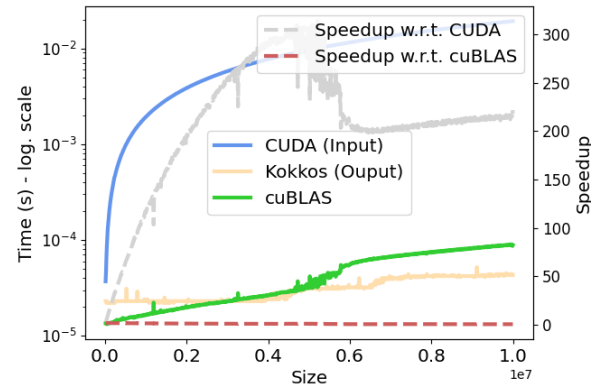


Figure 5: Performance comparison of a non-optimized CUDA code (ChatHPC for Kokkos input) that computes a dot-product and the Kokkos code generated by ChatHPC for Kokkos. Additionally, we include the performance of an NVIDIA cuBLAS sdot code.

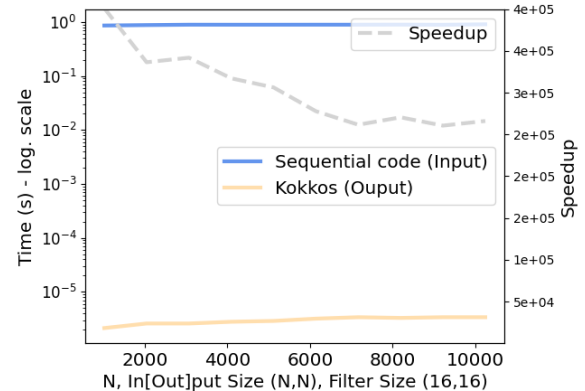


Figure 6: Performance comparison of a sequential code that implements an AI convolution (ChatKokkos input) and the Kokkos code generated by ChatKokkos.

produce vast amounts of data and require scalable I/O capabilities. To address this growing problem, the ADIOS parallel I/O library was completely rewritten as ADIOS2 to cope with extreme-scale I/O. ADIOS2 is written in C++11 and provides bindings for C, Fortran, Python, Matlab, and Julia to enable high-performance writes to the parallel file system and in-memory communication pipelines [35]. The ADIOS2 programming model expresses code in terms of scientific variables, attributes, and operations, thus separating application syntax from configuration settings (e.g., back ends) to enable performance-portable I/O across multiple physical data transport media. Given the numerous ADIOS2 applications, we targeted ADIOS for an AI assistant so application developers and users can simplify their interactions with ADIOS2 and improve their productivity when building their application workflows.

```

1 adios2::ADIOS adios("adios2_config.yaml", mpiComm);
2 adios2::IO io = adios.DeclareIO("WriteIO");
3 adios2::Variable<double> varU =
4   io.DefineVariable<double>("U", shape, start, count);
5 adios2::Operator Zfp =
6   adios.DefineOperator("Zfp", adios2::ops::LossyZFP);
7 varU.AddOperation(Zfp, {{ adios2::ops::zfp::key::rate, "9" }});
8 adios2::Engine bpWriter =
9   io.Open("output.bp", adios2::Mode::Write);
10 while( run_simulation ){
11   bpWriter.BeginStep();
12   bpWriter.Put<double>(varU, U.data());
13   ...
14   bpWriter.EndStep();
15 }
16 bpWriter.Close();

```

Figure 7: Basics of ADIOS2 parallel I/O programming model for writing a compressed scientific variable, U , using Zfp.

Table 2: ChatADIOS2 testing (percentage of prompts with correct responses).

Context	CODE LLAMA	ChatADIOS2 Initial	ChatADIOS2 Refinement	ChatGPT
Introduction	0%	77%	100%	33%
Variable Definition	0%	50%	100%	33%
Data Compression	0%	25%	100%	0%
Parallelization	0%	0%	100%	100%

Data: We focus on four common aspects (contexts) of user interaction with the ADIOS2 public APIs: (1) simple Q&A for an introduction to the library; (2) tasks related to interacting with the ADIOS2 APIs, including compression with common lossy Zfp [36, 37]; and (3) porting serial POSIX I/O to ADIOS2 for parallel performance. To test the capabilities of the fine-tuned model, we use prompts that combine text with code snippets that use the ADIOS2 C++ APIs.

Analysis: Our initial testing (Table 2) confirms three things: (1) The Code Llama base model has no knowledge of ADIOS2, returns erroneous answers, and hallucinates in almost all cases. (2) ChatGPT is aware of ADIOS2, and we assume this is because of publicly available information online, but basic to slightly more advanced coding tasks are not well understood, which results in seemingly correct code with several flaws (e.g., race conditions, wrong API names). (3) Fine-tuning Code Llama with added context (ChatADIOS2) could provide a reliable AI assistant for ADIOS2.

Refinements are performed to understand the scope of how much ChatADIOS2 can adapt to the patterns found in the ADIOS2 APIs. As seen in Table 2, introductory questions score high because of the simple nature of the variants in the fine-tuned data. However, when responses are included implicitly in other responses, ChatADIOS2 fails and requires explicit refinement. For example, the answer to “Which bindings are supported in ADIOS?” is within “What is ADIOS?” We also ask questions that are specific to defining variables (e.g., define an ND variable [3D, 5D, etc.]) even though ChatADIOS2 knows only of 2D variables. Unsurprisingly, ChatADIOS2 can understand this simple pattern in the DefineVariable function, but only for global variables because the more common use cases have higher representation in the original fine-tuning dataset. In contrast, responses for ADIOS2’s local variable functionality are mixed with global variables in all cases, thereby requiring explicit refinement.

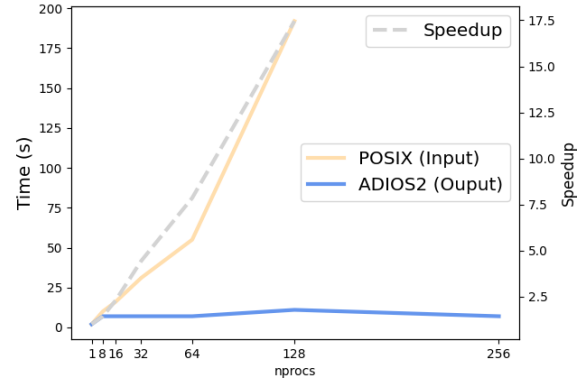


Figure 8: Performance gains for translating an MPI_Gather serial POSIX I/O code to ADIOS2 using ChatADIOS2.

Next, we formulate questions related to compression methods in the ADIOS2 API. Different compression libraries require specific sets of parameters and result in minimally overlapping learning contexts. For example, as seen in Line 7 of Figure 7, no matter how well ChatADIOS2 understands the adjustments for Zfp compression, it still needs to be fine-tuned for other methods (e.g., BZip, SZ) and their respective terminologies.

A common case is to port serial I/O codes to ADIOS2. The typical pattern is to send all the data to MPI (Message Passing Interface) rank 0 and write a raw data file using POSIX APIs or those implemented in C or C++. This pattern also stresses the memory in rank 0, and POSIX limits its write size to 2 GB buffers, thereby making it very inefficient for larger data sizes. ChatADIOS2 is fine-tuned to translate MPI_Gather typed memory into proper ADIOS2 global variables. For validation, we train using MPI_INT but request a double variable that will be later compressed by Zfp. ChatADIOS2 issues a code similar to that (Figure 7) without any input of ADIOS2 code in the prompt.

We also investigate the transformation of serial codes to ADIOS2 to validate the ChatADIOS2-generated codes on a real system like the Frontier supercomputer and its Orion parallel file system [38] (Figure 8). In all cases, we use ADIOS2 v2.10, Frontier’s Cray-MPICH, and GCC 13. Both the original POSIX and the AI-assisted ChatADIOS2 codes write 100 million doubles (~800 MB) per MPI process (nproc), hence we test for weak scaling by increasing the number of nodes and MPI processes (8 per node to match the number of GPU compute dies available to applications using ADIOS2 on Frontier). As expected, the POSIX implementation stops at 128 processes due to memory requirements, whereas the ADIOS2 code guarantees parallel I/O scalability beyond this number.

We also validate the compression rates of the generated code on Frontier’s Orion (Figure 9). The applied Zfp compression (using the latest Zfp v1.0.1), obtained through ChatADIOS2 by generating an ADIOS2 operator, results in data sizes compressed nearly 7× for the selected Zfp rate of 9 and the characteristics of the data itself (simple range from 1 to N).

We add the modality of runtime information for performance and compression rates for validation. Overall, because of the simplicity

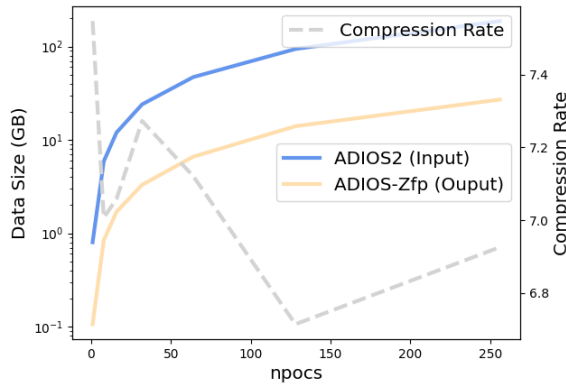


Figure 9: Data compression ratios for enabling Zfp with ChatADIOS2.

of the ADIOS2 API, ChatADIOS2 can generate useful code snippets to aid in the development and productivity of HPC software that requires parallel I/O capabilities.

5.3 ChatMAGMA

In HPC, math libraries are one of the most important components of the software stack and provide optimized routines for various mathematical operations, including linear algebra, matrix manipulations, and fast transforms.

The portable and open-source Matrix Algebra on GPU and Multicore Architectures (MAGMA) [7] math library is a collection of more than 400 routines for heterogeneous computing. The MAGMA package [39] supports interfaces for standard linear algebra packages (e.g., LAPACK, BLAS) to enable computational scientists to easily port any linear algebra-reliant software component to a modern hybrid computing system consisting of many-core CPUs and multiple GPUs/coprocessors. Unlike vendor-specific libraries, MAGMA can simultaneously exploit both CPUs and GPUs. MAGMA is also compatible with many HPC (CPU and GPU) architectures.

Although most of these libraries try to follow the same standard BLAS/LAPACK specification [40, 41], there are important differences in their APIs (Figure 10). These differences stem from the pursuit of better performance on modern architectures or from commercial decisions that make applications compatible with only one specific architecture. This makes porting and optimizing applications a costly effort. ChatMAGMA focuses on easing this burden by using Code Llama to port applications that use vendor-specific or legacy sequential codes to equivalent codes that use the MAGMA library.

Data: Math libraries provide a separate implementation for each precision of each routine: single, double, single complex, and double complex or even half precision or mixed precision. The syntax and parameters are usually very similar between different routines (e.g., `potrf` for Cholesky factorization and `potrs` for the Cholesky solver). We can leverage the similarities in the syntax to minimize the size of the initial training data. We use a small subset of representative applications that use level-3 BLAS (e.g., matrix-matrix

Translate the next Intel MKL (CPU-only) `dgetrf` code:

```
1 // Intel MKL (CPU-only) dgetrf code
2 double *A=(double *)mkl_malloc(n*n*sizeof(double),64);
3 int *ipiv=(int *)mkl_malloc(n*sizeof(int),64);
4 LAPACK_dgetrf(LAPACK_ROW_MAJOR,n,n,A,n,ipiv);
5 mkl_free(A);
6 mkl_free(ipiv);
```

Translate the next NVIDIA cuSolver (GPU-only) `dgetrf` code:

```
1 // NVIDIA cuSolver (GPU-only) dgetrf code
2 int lwork=0;
3 double *dwork;
4 int *dipiv;
5 double *dinfo;
6 cusolverDnHandle_t handle;
7 cusolverDnCreate(&handle);
8 double *hA=(double *)malloc(n*n*sizeof(double));
9 cusolverDnDgetrf_bufferSize(handle,n,n,dA,n,&lwork);
10 cudaMalloc((void **)&dA,n*n*sizeof(double));
11 cudaMalloc((void **)&dwork,sizeof(double)*lwork);
12 cudaMalloc((void **)&dipiv,n*sizeof(int));
13 cudaMalloc((void **)&dinfo,sizeof(int));
14 cudaMemcpy(dA,hA,n*n*sizeof(double),cudaMemcpyHostToDevice);
15 cusolverDnDgetrf(handle,n,dA,n,dwork,dipiv,dinfo);
16 cudaMemcpy(hA,dA,n*n*sizeof(double),cudaMemcpyDeviceToHost);
17 cudaFree(dA);
18 cudaFree(dwork);
19 cudaFree(dipiv);
20 cudaFree(dinfo);
21 free(hA);
22 cusolverDnDestroy(handle);
```

ChatMAGMA output:

```
1 // MAGMA (CPU+GPU) dgetrf code
2 magma_queue_t queue;
3 magma_device_t cdev;
4 magma_getdevice(&cdev);
5 magma_queue_create(cdev,&queue);
6 double *hA;
7 magmaDouble_ptr dA;
8 magma_int_t *ipiv;
9 magma_int_t info;
10 magma_dmalloc_cpu(&hA,n*n);
11 magma_dmalloc(&dA,n*n);
12 magma_imalloc_cpu(&ipiv,n);
13 magma_dsetmatrix(n,n,hA,n,dA,n,queue);
14 magma_dgetrf_gpu(n,n,dA,n,ipiv,&info);
15 magma_dgetmatrix(n,n,dA,n,hA,n,queue);
16 magma_free(dA);
17 magma_free_cpu(hA);
18 magma_free_cpu(ipiv);
19 magma_queue_destroy(queue);
```

Figure 10: Example of two conversations (prompts) with ChatMAGMA.

multiplication, triangular solver) and LAPACK (e.g., Cholesky, LU factorization) routines, all of which use only double-precision. We use only data related to the Intel MKL (CPU-only) and NVIDIA cuBLAS (GPU-only) math libraries. We do not provide any data for the AMD hipBLAS libraries. Due to the repeating patterns found in math libraries, we use only a dozen prompts for the initial fine-tuning, and this initial training dataset is one of the smallest used for this work.

Analysis: After the initial fine-tuning, we ask ChatMAGMA to provide MAGMA codes from BLAS and LAPACK applications that use vendor-specific libraries but were not used for the initial training-data (e.g., applications that do not use double precision or use different BLAS or LAPACK factorizations/solvers not used for fine-tuning). Although the results are similar to the ones provided by other AI assistants described in this paper, ChatMAGMA provides relatively high-quality MAGMA codes. Actually, the codes

Table 3: ChatMAGMA testing (percentage of prompts with correct responses). [*] No data from AMD’s math libraries (e.g., hipBLAS and hipSolver) was used for the fine-tuning (e.g., self-learning).

Context	Code Llama	ChatMAGMA Initial	ChatMAGMA Refinement	ChatGPT
Intel	0.00%	37.50%	93.00%	0.00%
NVIDIA	0.00%	50.00%	96.50%	0.00%
AMD*	0.00%	45.30%	95.30%	0.00%

ported by ChatMAGMA are either 100% correct or very close to correct (more than 90% of the code was correct). In fact, the few errors that did occur are mainly found in the wrong use of some very specific MAGMA routines (e.g., use of two MAGMA queues for the potrs [Cholesky] solver instead of one), MAGMA-specific data type (e.g., use of the MagmaDouble type for single-precision codes), or functions (e.g., using specific double-precision data transfer primitives, magma_dset, for single-precision codes, magma_sset). This high-quality result simplifies the identification of learning gaps and the addition of the complementary information (i.e., refinement) to the initial training data. Notably, for both the initial training data and refinement, we did not use any data related to AMD’s math libraries.

Table 3 includes results for the base Code Llama and ChatGPT models. Although ChatGPT provided much better results (MAGMA codes) than the non-fine-tuned Code Llama base model, the generated codes were not fully compatible with the MAGMA API. Actually, none of the reference models could provide one fully correct MAGMA code. Depending on the case, the incorrect parts of the code ported by ChatGPT-4o varied by 20%–50% (e.g., wrong use of MAGMA queues, using APIs not provided by MAGMA).

After another fine-tuning iteration (refinement) and adding about twice the amount of data versus the initial training data to fill the learning gaps found during the first test, ChatMAGMA further increased its level of trustworthiness. Also, although no data about AMD math libraries was included for fine-tuning, we see equivalent results in portability for AMD libraries as we did for the other vendor-specific libraries.

5.4 ChatIRIS

The IRIS runtime is designed to provide seamless execution on heterogeneous multidevice and multivendor systems [42, 43]. The IRIS task-based programming model (Figure 11) allows users to express computation in terms of tasks and memory objects that are architecture-agnostic. At run time, IRIS schedules concurrent tasks on a set of heterogeneous devices, and each task invokes appropriate kernels for the underlying architecture. IRIS supports CPUs, GPUs, field-programmable gate arrays, and digital signal processors from major vendors, and it can schedule kernels written in CUDA, HIP, OpenCL, OpenACC, OpenMP, and Level Zero. An important feature of the IRIS runtime is its virtual shared device memory, which allows the runtime to track the location of the most updated memory. Owing to the memory abstraction provided by IRIS (Figure 11), users do not need to specify any data movement because the runtime does it for them. Moreover, IRIS conducts dataflow analysis to find concurrency by constructing a directed

acyclic graph and can simultaneously execute independent tasks on multiple devices. To provide portability to a multi-accelerator environment, IRIS divides its code into task code and kernel code. Even with the robust feature set provided by the IRIS runtime, implementing HPC codes using task-based programming models can be a challenge. ChatIRIS aims to mitigate this challenge by providing users with automatic generation of reliable task code.

Translate the next CUDA code to IRIS:

```

1 alpha = 2.0;
2 X = (float *) malloc(SIZE * sizeof(float));
3 Y = (float *) malloc(SIZE * sizeof(float));
4 Z = (float *) malloc(SIZE * sizeof(float));
5 for (int i = 0; i < SIZE; i++) { X[i] = i; Y[i] = i;}
6 cudaMalloc((void**)&d_x, SIZE * sizeof(float));
7 cudaMalloc((void**)&d_y, SIZE * sizeof(float));
8 cudaMalloc((void**)&d_z, SIZE * sizeof(float));
9 // Initialization of arrays
10 cudaMemcpy(d_x, x, SIZE * sizeof(float), cudaMemcpyHostToDevice);
11 cudaMemcpy(d_y, y, SIZE * sizeof(float), cudaMemcpyHostToDevice);
12 int threads = 512;
13 int blocks = floor(SIZE/threads) + 1;
14 saxpy<<<blocks, threads>>>(alpha, d_x, d_y, SIZE);
15 cudaDeviceSynchronize()
16 sdot<<<blocks, threads>>>(d_x, d_x, SIZE);
17 cudaDeviceSynchronize()
18 cudaMemcpy(z, d_z, SIZE * sizeof(float), cudaMemcpyDeviceToHost);
19 cudaFree(d_x);
20 cudaFree(d_y);
21 cudaFree(d_z);
22 free(X);
23 free(Y);
24 free(Z);

```

ChatIRIS output:

```

1 // Architecture-agnostic IRIS task code
2 alpha = 2.0;
3 iris_init(&argc, &argv, 1);
4 iris_mem iris_x;
5 x = (float *) malloc(SIZE * sizeof(float));
6 iris_data_mem_create(&iris_x, x, SIZE * sizeof(float));
7 iris_mem iris_y;
8 y = (float *) malloc(SIZE * sizeof(float));
9 iris_data_mem_create(&mem_Y, Y, SIZE * sizeof(float));
10 iris_mem iris_z;
11 z = (float *) malloc(SIZE * sizeof(float));
12 iris_data_mem_create(&mem_z, z, SIZE * sizeof(float));
13 // Initialization of arrays
14 iris_graph graph;
15 iris_graph_create(&graph);
16 iris_task saxpy_task;
17 iris_task sdot_task;
18 iris_task_create(&saxpy_task);
19 iris_task_create(&sdot_task);
20 void* saxpy_params[3]={&alpha, &iris_x, &iris_y};
21 int saxpy_params_info[3]={sizeof(A), iris_w, iris_r};
22 iris_task_kernel(task0, "saxpy", 1, NULL, &SIZE, NULL, 3,
23     saxpy_params, saxpy_params_info);
24 void* sdot_params[2]={&iris_z, &iris_x};
25 int sdot_params_info[2]={iris_w, iris_r};
26 iris_task_kernel(task1, "sdot", 1, NULL, &SIZE, NULL, 2,
27     sdot_params, sdot_params_info);
28 iris_graph_task(graph, saxpy_task, iris_default, NULL);
29 iris_graph_task(graph, sdot_task, iris_default, NULL);
30 iris_graph_submit(graph, iris_any, 1);
31 free(X);
32 free(Y);
33 free(Z);
34 iris_finalize();

```

Figure 11: Example of a conversation with ChatIRIS.

Data: To ensure portability on a multi-accelerator platform, IRIS provides architecture-agnostic task code. The objective of ChatIRIS is to take the input of architecture-specific code (e.g., CUDA, HIP) and provide an architecture-agnostic IRIS task code while keeping the host code intact. These transformations focus on IRIS memory

Table 4: ChatIRIS testing (percentage of prompts with correct responses). [*]No data from the HIP programming model was used for the fine-tuning (e.g., self-learning)

Context	Code Llama	ChatIRIS	ChatGPT
Documentation	0.00%	95.00%	70.00%
OpenMP → IRIS task code	0.00%	82.50%	10.00%
CUDA → IRIS task code	0.00%	90.00%	50.00%
HIP* → IRIS task code	0.00%	81.66%	10.00%

object creation, task creation, how the task is made part of the graph, and graph submission (Figure 11). Notably, architecture-specific device memory creation and data transfer are completely omitted in the IRIS code because IRIS abstraction performs device memory creation and data movement between devices autonomously and transparently at run time. The data is divided into three contexts: Documentation, OpenMP to IRIS portability, and CUDA to IRIS portability. Documentation provides information about the IRIS runtime.

Analysis: After the initial fine-tuning, we ask ChatIRIS about Documentation and converting OpenMP, CUDA, and HIP code to IRIS task code. Note that no HIP codes are used during the fine-tuning. We compared the results between Code Llama, ChatGPT, and ChatIRIS (Table 4). Although the Code Llama base model provides a poor result, ChatGPT provides fairly accurate answers about IRIS documentation. For code transformation and portability, ChatIRIS is trained on variants of basic operations on 1D arrays (Figure 11). More complex codes that implement AI convolutions and computations on 2D matrices are used as part of the evaluation data. With minimal training, ChatIRIS provides nearly correct IRIS task code from a set of architecture-specific codes. Although ChatIRIS is not fine-tuned for HIP codes, it shows remarkable capabilities when porting HIP codes to IRIS. ChatGPT and Code Llama do not have much success in converting architecture-specific codes to IRIS. However, ChatGPT generates some correct IRIS API calls. Although different learning data (e.g., CUDA, HIP, OpenMP, IRIS source codes) is used for different capabilities and for fine-tuning and testing, the recurring and repeating patterns found in transforming architecture-specific codes to IRIS code may provide high levels of trustworthiness without refinement.

5.5 ChatTAU

TAU [8] provides a comprehensive set of tools for profiling, trace generation, and visualization of performance data for parallel applications, making it one of the most popular HPC performance measurement and analysis tools. TAU supports various HPC programming models and runtimes (e.g., MPI, OpenMP, CUDA, HIP, OpenCL) and provides seamless support for hybrid models (MPI + OpenMP, MPI + CUDA). TAU has robust profile analysis tools (both CLI and GUI) and can generate traces in different formats to create visualizations using different trace viewer tools (e.g., Vampir). Given its popularity and utility, we targeted TAU for an AI assistant. ChatTAU helps users choose the correct TAU options and generate the proper profiles for their performance monitoring needs.

Data: ChatTAU is trained primarily on the TAU documentation. We also curated a relatively small dataset of profiling and tracing

options in TAU. This smaller dataset contains questions about build instructions and profiling HPC programming models in solo and hybrid mode and creating visualizations (Figure 12).

Analysis: In general, for building and profiling in TAU, ChatGPT provides good answers owing to TAU’s extensive public documentation, which was collected over more than two decades and includes contributions from a wide user base. We study a total of 16 cases (testing data) that represent some of the most commonly used TAU configurations for the current standards. ChatGPT provides good instructions on how to build TAU for profiling C, C++, Fortran, and Python code. Moreover, it provides correct results about how to visualize profiles and traces generated by TAU. However, when it comes to the HPC programming models (e.g., MPI, CUDA, HIP, OpenMP), ChatGPT consistently provides wrong results about how to correctly build TAU and generate profiles. ChatGPT provides correct responses 56.25% of the time based on our testing data and is generally imprecise and sometimes erroneous when dealing with configurations that involve HPC programming models.

ChatTAU mitigates these deficiencies and provides the expertise needed for all building and profiling cases, including those that involve HPC configurations. Built on top of Code Llama, which does not have expertise in TAU, ChatTAU is fine-tuned to assist in building and profiling specific HPC configurations using different HPC programming models and runtimes. An example is shown in Figure 12, in which ChatTAU provides correct instructions on building and profiling by using TAU for a CUDA + MPI configuration. ChatTAU is aware that TAU uses the `cupti` interface for those configurations that involve CUDA. Other examples include AMD GPUs and MPI, the building and profiling instructions for which involve ROCm- and MPI-specific parameters, respectively. Although seemingly minor, in practice, the use of correct instructions and parameters is vital for TAU to generate the proper performance data.

How can I build TAU for profiling an MPI + CUDA application?

```

1 Clone the TAU repo:
2 git clone https://github.com/UO-OACISS/tau2.git
3 Build TAU using this option:
4 ./configure -mpi -cuda=cuda_path -prefix=install_path;
5 For profiling MPI + CUDA code:
6 tau_exec -T cupti -cupti ./program

```

Figure 12: Example of a conversation with ChatTAU.

5.6 ChatPORT

ChatPORT is a fine-tuned assistant designed for porting HPC codes after expert-guided training of a base model. We use Code Llama here. The main objectives of ChatPORT are to (1) translate low-level, hardware-specific code (e.g., CUDA) to portable, device-centric models such as SYCL and OpenMP and (2) establish the mechanisms to understand the amount and type of data needed to create capabilities (CUDA translations) in modern LLMs. In contrast to the AI assistants presented earlier, we use real-world data rather than synthetic data for ChatPORT’s training material, including codes extracted from a large collection of heterogeneous computing benchmarks (HeCBench) [44, 45] for CUDA, HIP, SYCL, and OpenMP.

Data: We use selected kernels from a variety of HeCBench benchmarks. Table 5 lists the different benchmark kernels used to train Code Llama to generate the fine-tuned LLM. These kernels represent real-world problems in various scientific domains. For each iteration of the training set, the kernels from the previous training set are also included. For example, Training Set 3 contains the kernels from Training Sets 1 and 2 in addition to the new kernels selected by expert-guided refinement from Training Set 3 (Table 5).

Table 5: HeCBench benchmarks used for ChatPORT.

Training Set	#Kernels	Benchmark (Kernels) Used
1	5	accuracy, geodesic, lr, maxpool3d, perplexity
2	10	damage, knn, heat2d, laplace3d, md
3	15	advCubatureHex3D, backprop, channelShuffle, lr, meanshift
4	22	chemv, clenergy, pathfinder, pointwise, swish, stddev, tissue

Analysis: Figure 13 shows the results of testing ChatPORT on 59 kernels extracted from other HeCBench benchmarks as evaluation data. Notably, these other kernels are not used in any of the training sets. After initial fine-tuning with Training Set 1, 13 (22%) and 11 (18%) CUDA kernels were well-translated to SYCL and OpenMP, respectively. These numbers are considerably better after fine-tuning with Training Set 4, when 42 (71%) and 48 (81%) of the CUDA kernels were correctly translated to SYCL and OpenMP, respectively.

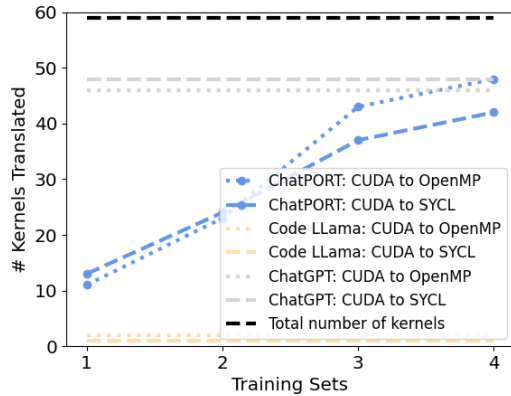


Figure 13: ChatPORT testing.

Our analysis of the CUDA-to-SYCL translation indicates that the major issues found correspond to static/dynamic allocation of shared memory, migration of 3D indices of the CUDA grid, and omission of the SYCL namespace for certain floating-point types (e.g., `float4`) and built-in arithmetic functions (e.g., from `sqrtf()` to `sycl::sqrt()`). For the CUDA-to-OpenMP translation, the incorrectly generated codes suffer from hallucinated directive combinations, incorrect loop conditionals, and the tendency to convert the first encountered loop body to an `omp target teams distribute`

`parallel` for construct. With successive training, we see a reduction across all the error categories, especially after Training Set 3, when there is a marked improvement in correctly offloading relevant loops and even identifying multiple offload regions within the same function.

The experimental results show that the sizes of the training sets are important for improving the success rates of code translation. We see a significant difference in the number of correctly translated codes for SYCL and OpenMP offload after Training Set 1. This is from the differences in the syntax of the target models (SYCL and OpenMP) versus CUDA. SYCL syntax is more similar to CUDA syntax, whereas OpenMP syntax is not as similar. A larger training set is required for CUDA-to-OpenMP translation. As shown in Figure 13, successive training sets allow ChatPORT to provide a comparable number of correctly translated codes for both OpenMP and SYCL. Using 22 kernels for training data and a few minutes of fine-tuning, ChatPORT provides much better results than Code Llama and competitive results versus ChatGPT 4o (no fine-tuning was done on these two models).

6 Observations and Future Focus

Table 6 lists the average percentage of prompts that were well answered for each of the ChatHPC AI assistants and for the reference models. Table 7 shows other metrics, including the data ratio used for training and testing for each AI assistant, as well as the increment (in terms of percentage) of the training data used for the different expert-supervised refinements and the impact of this on the performance (i.e., the increment of well-answered prompts after the refinements).

Table 6: Average percentage of working answers from each testing dataset after initial training and refinement.

Assistant	Code Llama	Initial	Refinement	ChatGPT
ChatHPC for Kokkos	7.34%	70.51%	85.75%	48.34%
ChatADIOS2	0.00%	40.87%	100.00%	25.00%
ChatMAGMA	0.00%	44.26%	94.93%	0.00%
ChatIRIS	0.00%	87.29%	-	35.00%
ChatPORT	2.54%	20.00%	76.00%	79.66%

With these results, we see the importance of involving human expertise in the creation (fine-tuning) of HPC-specific capabilities on pretrained and relatively small LLMs. With the exception of ChatIRIS, which provides very high levels of trustworthiness with no additional refinement, the expert-supervised tuning provides notable improvements in the performance of the AI assistants. With relatively modest computational cost and effort, ChatHPC provides either competitive or better performance than large state-of-the-art LLMs.

Although every AI assistant is different, in most cases, the data used for testing is larger than the data used for fine-tuning. Notably, in many cases, the testing data includes capabilities that are not included in the the fine-tuning (self-learning); in other words, the AI assistants are tested against data they have not seen. In general, the additional data used for refinements increases the correctness levels in terms of percentage, with the exception of ChatPORT, for which real data from the HeCBench suite was used instead of synthetic data.

Table 7: Left: Ratio of prompts used for training and testing before and after refinement (e.g., for ChatKokkos, during the initial training, 48% of all the prompts were used for training, and the rest [52%] were used for testing). Right: Percentage of prompts added to initial training data during the refinement and the increment of working answers after the refinement.

Metrics	Training::Testing Ratio		Increment (%) Ratio	
Assistant	Initial	Refinement	Training Data	Correctness
ChatHPC for Kokkos	48::52	54::46	12.33%	17.77%
ChatADIOS2	29::71	32::68	37.50%	62.85%
ChatMAGMA	36::64	46::54	51.72%	53.37%
ChatIRIS	59::41	-	-	-
ChatPORT	8::92	27::73	77.27%	76.68%

Here, we summarize several observations (Obs.) from our evaluation and discuss potential improvements for future work.

Obs. 1 – Fine-tuning: We demonstrated the effectiveness of fine-tuning pretrained and relatively small LLMs (7-billion parameter Code Llama model) by using expert-supervised data as a cost-effective approach to rapidly creating trustworthy HPC capabilities with high levels of correctness (up to more than 90% compared with state-of-the-art LLMs such as OpenAI’s 1.8-trillion parameter ChatGPT 4o model).

Obs. 2 – Trustworthiness: Fine-tuning a pretrained Code Llama model for very specific HPC targets by using very precise data (e.g., source codes) for both fine-tuning and testing may produce very high levels of trustworthiness even without further refinements. This is the case for some capabilities created with ChatADIOS2, ChatTAU, and ChatIRIS.

Obs. 3 – Overfitting: The lack of sufficiently diverse information or data in those domains, particularly niche or sparsely documented ones, may result in overfitting, which makes the AI assistants more deterministic and less creative. To avoid this, it is important to balance the amount of information provided for the training with the existing information. On the other hand, the impact of overfitting in this particular scenario—that is, creating highly specialized AI assistants for a particular task—is not necessarily a negative.

Obs. 4 – Self-learning: Because of the recurring repetitive patterns found in multiple HPC cases, AI assistants can learn new capabilities, such as portability of AMD math libraries (ChatMAGMA), translation of CUDA or HIP codes (ChatHPC for Kokkos and ChatIRIS), or N -dimensional data definition (ChatADIOS2), without being provided specific training data for those cases, thereby reducing the required size of the training datasets.

Obs. 5 – Synthetic or real data: There is no clear answer about the best kind of data to use for training and testing AI assistants. Although the synthetic data provided by experts can help fine-tune LLMs for new capabilities with less data by focusing specifically on the capabilities of interest, real data (e.g., applications, benchmarks, repositories) can provide a more realistic perspective about the capabilities needed by the HPC community. However, when using synthetic data, the experts have more control over the capabilities being trained into the different AI assistants. Achieving this level of control is more challenging and time-consuming when using real data and may add repetitive or redundant data to the training.

Obs. 6 – Code completeness: The source codes used for both fine-tuning and testing must be self-contained and readable, prioritizing readability over brevity. This helps AI assistants learn about the patterns and code transformations required for the different capabilities to be created.

Obs. 7 – Expert-in-the-loop: Performance of AI assistants improves considerably when human expertise is integrated into the fine-tuning process to create the training data and identify learning-gaps, although experts may not be always available.

Obs. 9 – Accessibility: ChatHPC infrastructure is accessible to everyone and requires only modest computational resources (a few minutes on a node with two NVIDIA GPUs). Also, we demonstrated that the data required for users to create new capabilities is relatively small (on the order of KB).

Obs. 10 – Simplify prompting: Communication with the ChatHPC-created AI assistants does not require any prior domain knowledge, thereby reducing the expertise required for the LLMs to provide good quality results. In fact, in multiple cases, the communication is reduced to only source code (e.g., porting CUDA kernels to SYCL by using ChatPORT). In other words, no prompt engineering is needed.

Obs. 11 – Impact on HPC: Apart from elevating the productivity of HPC software by using AI assistants in critical tasks (parallelization, portability, optimization, scalability, and instrumentation), the AI assistants can optimize codes from those passed as input to achieve important speedups (of up to 300×) and scalability improvements (of up to 17.5×) for different domains.

Obs. 12 – Leveraging HPC efforts: ChatHPC benefits from the important efforts made by the HPC community in the past decade to provide highly productive and portable solutions (e.g., Kokkos, ADIOS2), and the code transformations made by the AI assistants provide the capabilities (scalability, speedup, portability) that modern HPC requires.

7 Conclusions

ChatHPC provides a productive and scalable environment for the creation of new capabilities in the HPC software ecosystem by delivering higher trustworthiness levels than state-of-the-art LLMs (Code Llama and the much larger ChatGPT model) in critical HPC tasks, including parallelization, portability, optimization, scalability, and instrumentation. With relatively small training datasets (just a few KB), ChatHPC can rapidly create trustworthy AI assistants for specific HPC functions—including programming models (Kokkos, IRIS, OpenMP, SYCL, CUDA), I/O (ADIOS2), math libraries (MAGMA), and performance profilers (TAU)—on very modest computational resources (one node with two NVIDIA GPUs) by fine-tuning the open-source Code Llama LLM. Moreover, the HPC community can easily extend the existing ChatHPC portfolio with different AI assistants without replicating the base LLM model, providing a truly accessible method for leveraging AI to facilitate HPC software development.

Acknowledgments

This research used resources of the Oak Ridge Leadership Computing Facility and the Experimental Computing Laboratory at the Oak

Ridge National Laboratory, which is supported by the Office of Science of the US Department of Energy under Contract No. DE-AC05-00OR22725. This material is based upon work supported by the US Department of Energy, Office of Science's Advanced Scientific Computing Research program as part of the Advancements in Artificial Intelligence for Science program, Ellora and Durban projects, the Next Generation of Scientific Software Technologies program, S4PST and PESO projects, and the Scientific Discovery through Advanced Computing (SciDAC) program, RAPIDS2 SciDAC Institute for Computer Science, Data, and Artificial Intelligence.

References

- [1] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. Canton Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 pages. doi:10.48550/arXiv.2308.12950
- [2] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhattele. 2024. HPC-Coder: Modeling Parallel Programs using Large Language Models. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. 1–12. doi:10.23919/ISC.2024.10528929
- [3] X. Ding, L. Chen, M. Emani, C. Liao, P.-H. Lin, T. Vanderbruggen, Z. Xie, A. Cerpa, and W. Du. 2023. HPC-GPT: Integrating Large Language Model for High-Performance Computing. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. Association for Computing Machinery, Denver, CO, USA, 951–960. doi:10.1145/3624062.3624172
- [4] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 pages. doi:10.48550/arXiv.2401.14196
- [5] Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Q. Dang, Nathan D. Ellingwood, Rahul Kumar Gayatri, Evan Harvey, Daisy S. Hollman, Dan Ibanez, Nevin Liber, Jonathan R. Madsen, Jeff Miles, David Poliakoff, Amy Powell, Sivasankaran Rajamanickam, Mikael Simberg, Dan Sunderland, Bruno Turkcin, and Jeremiah J. Wilke. 2022. Kokkos 3: Programming Model Extensions for the Exascale Era. *IEEE Trans. Parallel Distributed Syst.* 33, 4 (2022), 805–817. doi:10.1109/TPDS.2021.3097283
- [6] Jungwon Kim, Seyoung Lee, Beau Johnston, and Jeffrey S. Vetter. 2021. IRIS: A Portable Runtime System Exploiting Multiple Heterogeneous Programming Systems. In *Proceedings of the 25th IEEE High Performance Extreme Computing Conference (HPEC '21)*. 1–8. doi:10.1109/HPEC49654.2021.9622873
- [7] Ahmad Abdelfattah, Natalie Beams, Robert Carson, Pieter Ghysels, Tzanio Kolev, Thomas Stitt, Arturo Vargas, Stanimire Tomov, and Jack Dongarra. 2024. MAGMA: Enabling exascale performance with accelerated BLAS and LAPACK for diverse GPU architectures. *The International Journal of High Performance Computing Applications* 38, 5 (2024), 468–490. arXiv:https://doi.org/10.1177/10943420241261960 doi:10.1177/10943420241261960
- [8] Allen D. Malony, Sameer Shende, Wyatt Spear, Chee Wai Lee, and Scott Biersdorff. 2011. Advances in the TAU Performance System. In *Tools for High Performance Computing 2011 - Proceedings of the 5th International Workshop on Parallel Tools for High Performance Computing, ZIH, Dresden, September 2011*, Holger Brunst, Matthias S. Müller, Wolfgang E. Nagel, and Michael M. Resch (Eds.). Springer, 119–130. doi:10.1007/978-3-642-31476-6_10
- [9] William F. Godoy, Norbert Podhorszki, Ruonan Wang, Chuck Atkins, Greg Eisenhauer, Junmin Gu, Philip Davis, Jong Choi, Kai Germaschewski, Kevin Huck, Axel Huebl, Mark Kim, James Kress, Tahsin Kurc, Qing Liu, Jeremy Logan, Kshitij Mehta, George Ostrouchov, Manish Parashar, Franz Poeschel, David Pugmire, Eric Suchyta, Keichi Takahashi, Nick Thompson, Seiji Tsutsumi, Lipeng Wan, Matthew Wolf, Kesheng Wu, and Scott Klasky. 2020. Adios 2: The Adaptable Input Output System. A framework for high-performance data management. *SoftwareX* 12 (2020), 100561. doi:10.1016/j.softx.2020.100561
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [11] Jonathan J. Webster and Chunyu Kit. 1992. Tokenization As The Initial Phase In NLP. In *14th International Conference on Computational Linguistics, COLING 1992, Nantes, France, August 23–28, 1992*. 1106–1110. https://aclanthology.org/C92-4173/
- [12] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2024. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. arXiv:2303.17568 [cs.LG] https://arxiv.org/abs/2303.17568
- [13] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason T. Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! *Trans. Mach. Learn. Res.* 2023 (2023). https://openreview.net/forum?id=KoFOg41haE
- [14] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xi-aoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR abs/2308.12950* (2023). arXiv:2308.12950 doi:10.48550/ARXIV.2308.12950
- [15] Ali TehraniJamsaz, Arijit Bhattacharjee, Le Chen, Nesreen K Ahmed, Amir Yazdankhsh, and Ali Jannesari. 2024. CodeRosetta: Pushing the Boundaries of Unsupervised Code Translation for Parallel Programming. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=v6hrg4O9gg
- [16] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *CoRR abs/2401.14196* (2024). arXiv:2401.14196 doi:10.48550/ARXIV.2401.14196
- [17] Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net. https://openreview.net/forum?id=R8sQPpGCv0
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguistics* 12 (2024), 157–173. doi:10.1162/TAACL_A_00638
- [19] Keita Teranishi, Harshitha Menon, William F. Godoy, Prasanna Balaprakash, David Bau, Tal Ben-Nun, Abhinav Bhattele, Franz Franchetti, Michael Franusich, Todd Gamblin, Georgios Georgakoudis, Tom Goldstein, Arjun Guha, Steven Hahn, Costin Iancu, Zheming Jin, Terry Jones, Tze Meng Low, Het Mankad, Narasinga Rao Miniskar, Mohammad Alaul Haque Monil, Daniel Nichols, Konstantinos Parasyris, Swaroop Pophale, Pedro Valero-Lara, Jeffrey S. Vetter, Samuel Williams, and Aaron Young. 2025. Leveraging AI for Productive and Trustworthy HPC Software: Challenges and Research Directions. arXiv:2505.08135 [cs.SE] https://arxiv.org/abs/2505.08135
- [20] Le Chen, Pei-Hung Lin, Tristan Vanderbruggen, Chunhua Liao, Murali Emani, and Bronis de Supinski. 2023. LM4HPC: Towards Effective Language Model Application in High-Performance Computing. In *OpenMP: Advanced Task-Based, Device and Compiler Programming*, Simon McIntosh-Smith, Michael Klemm, Bronis R. de Supinski, Tom Deakin, and Jannis Klinkenberg (Eds.). Springer Nature Switzerland, Cham, 18–33.
- [21] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhattele. 2024. HPC-Coder: Modeling Parallel Programs using Large Language Models. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. 1–12. doi:10.23919/ISC.2024.10528929
- [22] Jeff Rasley, Samyann Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 3505–3506. doi:10.1145/3394486.3406703
- [23] Christian Munley, Aaron Jarmusch, and Sunita Chandrasekaran. 2024. LLM4VV: Developing LLM-driven test suite for compiler validation. *Future Generation Computer Systems* 160 (2024), 1–13. doi:10.1016/j.future.2024.05.034
- [24] Bin Lei, Caiwen Ding, Le Chen, Pei-Hung Lin, and Chunhua Liao. 2023. Creating a Dataset for High-Performance Computing Code Translation using LLMs: A

- Bridge Between OpenMP Fortran and C++. In *IEEE High Performance Extreme Computing Conference, HPEC 2023, Boston, MA, USA, September 25–29, 2023*. IEEE, 1–7. doi:10.1109/HPEC58863.2023.10363534
- [25] Anthony Zhou, Linnia Hawkins, and Pierre Gentine. 2024. Proof-of-concept: Using ChatGPT to Translate and Modernize an Earth System Model from Fortran to Python/JAX. *CoRR* abs/2405.00018 (2024). arXiv:2405.00018 doi:10.48550/ARXIV.2405.00018
- [26] William Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey Vetter. 2023. Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation. In *Proceedings of the 52nd International Conference on Parallel Processing Workshops* (, Salt Lake City, UT, USA,) (*ICPP Workshops '23*). Association for Computing Machinery, New York, NY, USA, 136–144. doi:10.1145/3605731.3605886
- [27] William F. Godoy, Pedro Valero-Lara, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter. 2024. Large language model evaluation for high-performance computing software development. *Concurr. Comput. Pract. Exp.* 36, 26 (2024). doi:10.1002/CPE.8269
- [28] Pedro Valero-Lara, Alexis Huante, Mustafa Al Lail, William F. Godoy, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter. 2023. Comparing Llama-2 and GPT-3 LLMs for HPC kernels generation. arXiv:2309.07103 [cs.SE]
- [29] Pedro Valero-Lara, William F. Godoy, Keita Teranishi, Prasanna Balaprakash, and Jeffrey S. Vetter. 2024. ChatBLAS: The First AI-Generated and Portable BLAS Library. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, Atlanta, GA, USA, November 17–22, 2024*. IEEE, 19–24. doi:10.1109/SCW63240.2024.00010
- [30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
- [31] 2025. Ollama. <https://ollama.com/>. [Online; accessed 18-March-2025].
- [32] 2025. Hugging Face. <https://huggingface.co/>. [Online; accessed 18-March-2025].
- [33] Pedro Valero-Lara, Seyong Lee, Marc González Tallada, Joel E. Denny, and Jeffrey S. Vetter. 2022. KokkACC: Enhancing Kokkos with OpenACC. In *9th Workshop on Accelerator Programming Using Directives, WACCPD@SC 2022, Dallas, TX, USA, November 13–18, 2022*. IEEE, 32–42. doi:10.1109/WACCPD56842.2022.00009
- [34] 2025. Kokkos. <https://kokkos.org/kokkos-core-wiki/index.html>. [Online; accessed 18-March-2025].
- [35] Franz Poeschel, Juncheng E, William F. Godoy, Norbert Podhorszki, Scott Klasky, Greg Eisenhauer, Philip E. Davis, Lipeng Wan, Ana Gainaru, Junmin Gu, Fabian Koller, René Wiedera, Michael Bussmann, and Axel Huebl. 2022. Transitioning from File-Based HPC Workflows to Streaming Data Pipelines with openPMD and ADIOS2. In *Driving Scientific and Engineering Discoveries Through the Integration of Experiment, Big Data, and Modeling and Simulation*. Jeffrey Nichols, Arthur ‘Barney’ Maccabe, James Nutaro, Swaroop Pophale, Pravallika Devineni, Theresa Ahearn, and Becky Verastegui (Eds.). Springer International Publishing, Cham, 99–118.
- [36] Peter Lindstrom. 2014. Fixed-Rate Compressed Floating-Point Arrays. *IEEE Transactions on Visualization and Computer Graphics* 20, 12 (2014), 2674–2683. doi:10.1109/TVCG.2014.2346458
- [37] Peter Lindstrom, Jeffrey Hittinger, James Diffenderfer, Alyson Fox, Daniel Osei-Kuffuor, and Jeffrey Banks. 2025. ZFP: A compressed array representation for numerical computations. *The International Journal of High Performance Computing Applications* 39, 1 (2025), 104–122. arXiv:https://doi.org/10.1177/10943420241284023 doi:10.1177/10943420241284023
- [38] Scott Atchley, Christopher Zimmer, John Lange, David Bernholdt, Veronica Mellesse Vergara, Thomas Beck, Michael Brim, Reuben Budiardja, Sunita Chandrasekaran, Markus Eisenbach, Thomas Evans, Matthew Ezell, Nicholas Frontiere, Antigoni Georgiadou, Joe Glenski, Philipp Grete, Steven Hamilton, John Holmen, Axel Huebl, Daniel Jacobson, Wayne Joubert, Kim McMahon, Elia Merzari, Stan Moore, Andrew Myers, Stephen Nichols, Sarp Oral, Thomas Papatheodore, Danny Perez, David M. Rogers, Evan Schneider, Jean-Luc Vay, and P. K. Yeung. 2023. Frontier: Exploring Exascale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) (*SC '23*). Association for Computing Machinery, New York, NY, USA, Article 52, 16 pages. doi:10.1145/3581784.3607089
- [39] 2025. MAGMA. <https://github.com/icl-utk-edu/magma>. [Online; accessed 18-March-2025].
- [40] Jack J. Dongarra. 2002. Basic Linear Algebra Subprograms Technical (BLAS) Forum Standard (2). *Int. J. High Perform. Comput. Appl.* 16, 2 (2002), 115–199.
- [41] Jack J. Dongarra and Piotr Luszczek. 2011. LAPACK. In *Encyclopedia of Parallel Computing*, David A. Padua (Ed.). Springer, 1005–1006. doi:10.1007/978-0-387-09766-4_152
- [42] J. Kim, S. Lee, B. Johnston, and J. S. Vetter. 2021. IRIS: A Portable Runtime System Exploiting Multiple Heterogeneous Programming Systems. In *IEEE High Performance Extreme Computing Conference (HPEC)*. doi:10.1109/HPEC49654.2021.9622873
- [43] N. R. Miniskar, S. Lee, J. Beau, A. Young, M. A. H. Monil, P. Valero-Lara, and J. S. Vetter. 2024. IRIS Reimagined: Advancements in Intelligent Runtime System for Task-Based Programming. In *Workshop on Asynchronous Many-Task Systems and Applications*. Springer Nature Switzerland Cham, 46–58.
- [44] Zheming Jin and Jeffrey S. Vetter. 2023. A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 325–327. doi:10.1109/ISPASS57527.2023.00041
- [45] 2025. HeCBench. <https://github.com/zjin-lcf/HeCBench>. [Online; accessed 18-March-2025].

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

A Overview of Contributions and Artifacts

The ChatHPC ecosystem is composed of (i) the ChatHPC library and (ii) a collection of AI assistants created by the ChatHPC library. We provide one repository that contains the ChatHPC library and the data (initial and refinement) to create (fine-tune) and test (verify and evaluate) ChatHPC for Kokkos.

A.1 Paper’s Main Contributions

- C₁** A lightweight software infrastructure (i.e., the ChatHPC library) that enables HPC experts to efficiently create and supervise trustworthy AI capabilities for critical HPC software tasks, thereby lowering the barrier to AI integration in HPC systems while maintaining reasonable computational requirements.
- C₂** A demonstration and evaluation of ChatHPC on important components of the HPC software stack, including programming systems (e.g., Kokkos, IRIS, SYCL, OpenMP, CUDA), scientific libraries (e.g., MAGMA), tools (e.g., TAU), and I/O (e.g., ADIOS2). We also show the challenges for the unique patterns and characteristics of each product. In this case, to keep the time and effort required to reproduce results at a reasonable level (less than one hour), we focus on the creation and evaluation of the ChatHPC for Kokkos AI assistant.

A.2 Computational Artifacts

A₁ ChatHPC-ChatHPCforKokkos-SC25, <https://doi.org/10.5281/zenodo.15226006>

Artifact ID	Contributions Supported	Related Paper Elements
A ₁	C ₁ and C ₂	Tables 1, 6, and 7

B Artifact Identification

B.1 Computational Artifact A₁

Relation To Contributions

ChatHPC for Kokkos is one of the AI assistants presented in the paper. Actually, it is the largest AI assistant created as part of the ChatHPC portfolio in terms of capabilities as well as training and testing data set size. To create ChatHPC for Kokkos, we must use the ChatHPC library. So, with this Artifact, we cover an important part of both contributions (e.g., C₁ and C₂) presented in our paper. To reduce the time needed to reproduce the results shown in the paper, we provide a subset of the training and testing data sets used for ChatHPC for Kokkos, in particular, we focus on the two main capabilities of that AI assistant; Parallelization and Translation. Training with a subset of the full data may lightly affect the results shown in Table 1, but the percentage of prompts well-answered are very similar, if not identical. The exact output from any given training run may vary slightly due to the random nature of the training algorithm, but the overall conclusions from the results will stay the same.

Expected Results

ChatHPC for Kokkos results should be very similar or even identical to the results shown in Table 1 by using the initial and refinement training data sets provided. Regarding the results for the other reference models, Code Llama and OpenAI GPT-4o, while the first one should provide the same results, the second one may provide different results as its capabilities may change over time.

Expected Reproduction Time (in Minutes)

The expected computational time of the A₁ Artifact (ChatHPC-ChatHPCforKokkos), fine-tuning the assistant on one two-GPU NVIDIA H100 node, is about 5–10 min, with the full training and evaluation process on both fine-tuned models, the base Code Llama model, and the comparison GPT-4o model, completing in under an hour. Using a platform other than one with two NVIDIA H100 GPUs is possible, as long as there is support for PyTorch. However, the time to make the fine-tuning can be very different depending on the resources used. The Artifact Setup is automated as part of the scripted demonstration flow. The Artifact Analysis does not require any additional computation and only takes a few minutes for the reviewer to look through the resulting assistant output as presented in markdown files to verify the improved responses of the fine-tuned model.

Artifact Setup (incl. Inputs)

Hardware. We made use of the resources provided by the Experimental Computing Laboratory (ExCL)¹ at Oak Ridge National Laboratory. Project and account applications are open for ExCL; however, using ExCL is not required to reproduce ChatHPC for Kokkos results. We do recommend leveraging a modern GPU system with at least 1 GPU (NVIDIA preferred). The NVIDIA GPUs tested in this work are Ampere A100 and Hopper H100.

Software. The ChatHPC ecosystem relies on (i) the 7 billion (~13 GB) Large Language Model (LLM) model Code Llama, which can be downloaded via <https://huggingface.co/meta-llama/CodeLlama-7b-hf>, (ii) the LoRA (low-rank adaptation) tool used for fine-tuning pretrained LLMs, and (iii) the open-source PyTorch Python framework used for building machine learning models. Specifically, for these last two dependencies, we use the versions packaged within the Hugging Face Transformers, PEFT, and Accelerate Python Packages. The version details of all the underlying Python packages are controlled transparently by the `pyproject.toml` Python project configuration file provided as part of the ChatHPC library.

Datasets / Inputs. We provide the data for fine-tuning (initial and refinement) and testing ChatHPC for Kokkos. These are part of the repository provided and can be found in the C2_Kokkos_Dataset folder.

Installation and Deployment. We use a set of scripts for the installation and deployment. The Artifact’s scripts depend on using Astral-uv² to build the Python virtual environment and run the

¹<https://excl.ornl.gov/> and <https://docs.excl.ornl.gov/>

²<https://docs.astral.sh/uv/>

software with all the correct dependencies in place. Astral-uv is an optimized and fast Python package and project manager, written in Rust, and, importantly for this artifact, useful for creating reproducible Python environments. A full list of the dependencies can be found in the `C1_ChathPC_Lib/ChatHPC-app-v25.4.1/pyproject.toml` Python project file. Astral-uv can be installed using `curl -LsSf https://astral.sh/uv/install.sh | sh`³ and once installed provides the needed `uv` command. The `uv` command is not needed directly, but it is a dependency of the bash scripts used to reproduce the results presented in this Artifact. As an example, the command

```
uv run --project C1_ChathPC_Lib/ChatHPC-app-v25.4.1 chathpc -h
```

uses `uv run` to ensure the Python environment is setup to provide access to the `chathpc` Command-Line-Interface (CLI) application. Additionally, this Artifact was developed on an Ubuntu 22.04.5 Long-Term-Support (LTS) system and should work on any modern Linux system.

Artifact Execution

The Artifact repository is structured such that the evaluation can be made by following these two simple steps.

- (1) Download the Code Llama base model weights. This can be done, automatically using the provided setup script, assuming the user's SSH key pair is registered with Hugging Face and access is granted to <https://huggingface.co/meta-llama/CodeLlama-7b-hf>. Alternatively, these weights can be manually downloaded from the Hugging Face website and placed in the base models directory at `basemodels/CodeLlama-7b-hf`.
- (2) Execute the provided bash scripts. The `run_all.sh` script is provided as a single top-level script and will call:
 - `1_setup.sh`, initial setup of the ChatHPC library application using `uv` to install/setup the Python environment and download the dependencies; PyTorch, LoRA, Code Llama, among others. As mentioned, Meta's license agreement must be accepted to download Code Llama.
 - `2_train.sh`, initial and refinement fine-tuning for the two versions of the ChatHPC for Kokkos assistant (e.g., initial and refinement) using the ChatHPC library.
 - `3_verify.sh`, verification for both fine-tuned assistants to check that the fine-tuning was successful.
 - `4_evaluate.sh`, evaluation of both fine-tuned assistants on testing data. This should provide the results shown in Table 1, 6, and 7.
 - `5_evaluate_baseline.sh`, conducts the same testing done in the previous step, but using the baseline models used in the paper (e.g., Code Llama and GPT-4o). This completes the results for the Table 1, 6, and 7 with baseline model results.

Note on script run order: `1_setup.sh` is a prerequisite to run any of the other steps as it sets up the environment. `2_train.sh` is required before running `3_verify.sh` or `4_evaluate.sh`. The `3_verify.sh` and `4_evaluate.sh` scripts do not depend on each other, and both use the trained

model created in `2_train.sh`. `5_evaluate_baseline.sh` only depends on `1_setup.sh` as it evaluates the baseline modes and not the trained model.

Artifact Analysis (incl. Outputs)

As output, ChatHPC generates reports (see Figure 14) as the output artifacts, which include the question (prompts) and the response generated by the AI assistant (e.g., ChatHPC for Kokkos in this case). The created output artifacts from training and evaluating the ChatHPC for Kokkos assistant are created and placed in the output and evaluation directories by the scripts. The main artifacts to review are the markdown files ending with the `.md` extension. These files can be opened with your text editor of choice, and they contain the structured results of the evaluation. For example, the `ChatHPCforKokkos_refinement_results.md` file contains the results from evaluating the refined ChatHPC for Kokkos assistant. See Figure 14 for an example of this evaluation output for the first test question. Saved results from running the artifact generation on our system are stored in `example_evaluation_output` to provide an example of the expected output after evaluating this artifact. Additionally, the expected console output from the `run_all.sh` script is provided in the `example_evaluation_output/run_all_output.txt` file.

```
1 # ChatHPC JSON to Markdown
2 ## Index 0
3
4 ### Context
5 OpenMP to Kokkos translation
6
7 ### Question
8 Can you translate this OpenACC code to Kokkos?
9 #pragma acc parallel loop
10 for (i = 0, i < n, i++) {
11     y[i] = a * x[i] + y[i];
12 }
13
14 ### Response
15 Kokkos::parallel_for("saxpy", Kokkos::RangePolicy<>(0, n),
16     KOKKOS_LAMBDA(const int i) {
17         y(i) = a * x(i) + y(i);
18     });
19 ...
```

Figure 14: Example evaluation output for the first test case for the ChatHPC for Kokkos refined assistant.

To reproduce the results shown in Tables 1, 6, and 7, one must evaluate the result generated by ChatHPC for Kokkos; in other words, one must evaluate whether the Kokkos code generated is correct and uses the proper Kokkos syntax. The tables show the percentage of prompts that were well-answered by ChatHPC for Kokkos. Note that we focus on the *Translation* and *Parallelization* contexts or capabilities in this analysis.

³<https://docs.astral.sh/uv/getting-started/installation/>

Artifact Evaluation (AE)

C.1 Computational Artifact A_1

Artifact Setup (incl. Inputs)

Artifact A_1 ChatHPC-ChatHPCforKokkos-SC25 is available from Zenodo (<https://doi.org/10.5281/zenodo.15226006>) or directly from GitHub (<https://github.com/ORNL/ChatHPC-ChatHPCforKokkos-SC25>). See the included README.md for a breakdown of the directory structure and an overview of the contained files.

The first step is to log on to a system with a GPU, preferably running Ubuntu 22.04.5 LTS or a newer LTS with a server-grade GPU, preferably an NVIDIA Ampere A100 or Hopper H100. Other operating systems and GPU architectures should also work, depending on their support with PyTorch and Hugging Face Python packages.

Next, the artifact will need to be downloaded to the system. The easiest way to do this is via git.

```
1 git clone git@github.com:ORNL/ChatHPC-ChatHPCforKokkos-SC25.git
```

Next, Astral-uv will need to be installed. The Artifact's scripts depend on uv⁴ to build the Python virtual environment and to run the software with all the correct dependencies installed. A full list of the dependencies can be found in the C1_ChatHPC_Lib/ChatHPC-app-v25.4.1/pyproject.toml file. Please install uv, using the standard instructions for installing uv⁵. Installation of uv on Linux is done with the following command.

```
1 curl -Lsf https://astral.sh/uv/install.sh | sh
```

The next step is to download the base LLM model. This can be done either by registering the system's SSH key with Hugging Face and requesting access to <https://huggingface.co/meta-llama/CodeLlama-7b-hf> to allow the scripts to automatically pull the base model's weights, or by navigating to <https://huggingface.co/meta-llama/CodeLlama-7b-hf>, downloading the model's weights, and placing them in at basemodels/CodeLlama-7b-hf.

Setup of the environment is done automatically using the included scripts. The full evaluation is flow found in run_all.sh; however, just the setup portion can be run with 1_setup.sh. The setup scripts provide error messages if the base model is not downloaded and notes about how to set the OPENAI_API_KEY if you optionally want to evaluate with the GPT-4o model. (Using an OpenAI API key incurs a cost and is thus not recommended or necessary to review the artifact, but it is included as an optional step for completeness.)

Artifact Execution

To run the complete workflow, run the run_all.sh base script, which will then call 1_setup.sh, 2_train.sh, 3_verify.sh, 4_evaluate.sh, and 5_evaluate_baseline.sh in order.

```
1 ./run_all.sh
```

An example of the expected output from running this script can be found in example_evaluation_output/run_all_output.txt.

⁴<https://docs.astral.sh/uv/>

⁵<https://docs.astral.sh/uv/getting-started/installation/>

These scripts make it easy to automate the reproduction of the results; however, the same results can be obtained by calling the chathpc CLI directly. See the annotated example below for what this would look like.

```
1 # For convenience install packages in a virtual environment and
  # load the environment. Doing so avoids the addition of the `uv
  # run --project C1_ChatHPC_Lib/ChatHPC-app-v25.4.1` command
  # prefix.
2 uv venv
3 source .venv/bin/activate
4 uv pip install ./C1_ChatHPC_Lib/ChatHPC-app-v25.4.1
5
6 ## 2_train.sh: Train the model
7 # Train the initial model
8 chathpc --config config_initial.json train
9 # Train the refined model
10 chathpc --config config_refinement.json train
11
12 ## 3_verify.sh: Verify the model trained successfully by testing
  # with the training dataset.
13 # Verify the initial model
14 chathpc --config config_initial.json verify
15 # Verify the refined model
16 chathpc --config config_refinement.json verify
17
18 ## 4_evaluate.sh: Evaluate the model on the testing dataset
19 # Test the initial model
20 chathpc --config config_initial.json test --save_results_file
  # evaluation/ChatHPCforKokkos_initial_results.json
  # C2_Kokkos_Dataset/kokkos_testing.yaml
21 # Convert the output json to markdown for easier viewing.
22 chathpc data-to-md evaluation/ChatHPCforKokkos_initial_results.
  # json > evaluation/ChatHPCforKokkos_initial_results.md
23 # Test the refined model
24 chathpc --config config_refinement.json test --save_results_file
  # evaluation/ChatHPCforKokkos_refinement_results.json
  # C2_Kokkos_Dataset/kokkos_testing.yaml
25 # Convert the output json to markdown for easier viewing.
26 chathpc data-to-md evaluation/ChatHPCforKokkos_refinement_results.
  # json > evaluation/ChatHPCforKokkos_refinement_results.md
27
28 ## 5_evaluate_baseline.sh: Evaluate the base model and optionally
  # an OpenAI model.
29 # Evaluate Code Llama base model.
30 chathpc --config config_initial.json base test --save_results_file
  # evaluation/code_llama_base_results.json C2_Kokkos_Dataset/
  # kokkos_testing.yaml
31 chathpc data-to-md evaluation/code_llama_base_results.json >
  # evaluation/code_llama_base_results.md
32 # Optional: Evaluate GPT-4o model
33 chathpc --config config_initial.json openai test --model gpt-4o --
  # save_results_file evaluation/openai_gpt-4o_base_results.json
  # C2_Kokkos_Dataset/kokkos_testing.yaml
34 chathpc data-to-md evaluation/openai_gpt-4o_base_results.json >
  # evaluation/openai_gpt-4o_base_results.md
```

Artifact Analysis (incl. Outputs)

The expected results can be found in the example_evaluation_output folder and the new results created by the scripts are found in evaluation. The output folder contains the training checkpoints, adapter model weights, and merged full model weights created by ChatHPC.

The easiest way to verify the output artifacts is to compare the example files found in example_evaluation_output with the new files created in evaluation. Although there might be some variation from run to run with the randomness in the training, you should find the new output files similar to the example output files. Manually verifying the results requires expertise with the Kokkos API, as you will need to look through the output markdown files and see if the LLM assistant correctly answered the questions given to it. If you tally up the correct and incorrect responses, you will generate a table similar to Table 1 in the paper.

Reproducibility Report

D Overview of Reproduction of Artifacts

The following table provides an overview of each computational artifact’s reproducibility status. Artifact IDs correspond to those in the AD/AE Appendices.

Artifact ID	Available	Functional	Reproduced
A_1	•	•	◦
Badge awarded	yes	yes	no

E Reproduction of Computational Artifacts

E.1 Timeline

The artifact evaluation was conducted from September 1, 2025, to Spetember 4, 2025.

E.2 Computational Environment and Resources

The experiments conducted for artifact evaluation were performed on two different computing environments. The first environment was a local workstation equipped with an NVIDIA RTX A6000 GPU providing 48 GB of VRAM. The second environment was the PlaFRIM (<https://www.plafrim.fr/>) cluster, where experiments were conducted on nodes equipped with both NVIDIA V100 GPUs and NVIDIA A100 GPUs (40 GB VRAM each).

E.3 Details on Artifact Reproduction

The procedures work as outlined in the AD/AE Appendices for the first environment with a NVIDIA RTX A6000. The execution script is very well detailed, and the user only needs to run a command to set up and launch the training algorithm. Initially, a memory error was encountered during training. Enabling the fragmentation option allowed the script to run to completion without reporting errors. However, the model was not actually trained: the script terminated quickly (within a few minutes instead of the expected one hour), and no output files were produced.

The second environment is more closely aligned with the hardware described in the article, as it consists of the cluster Plafrim equipped with both V100 and A100 GPUs. The article used H100 and A100 GPUs. For the artifact evaluation, the software was installed and tested separately on nodes equipped with V100 and A100 (40 GB VRAM for both) cards.

- **Versioning issues:** The pyproject.toml dependency file must be adjusted if the cluster environment does not support the latest versions of the required packages, since the software relies on the most recent Linux wheels. Depending on the changes, the 1_setup.py script may also need to be modified.
- **Data accessibility:** Access to the data requires prior authorization from the host on Hugging Face. If the cluster does not support the use of an SSH key to fetch the model weights directly, the files must be downloaded on a local machine and then transferred to the cluster. Since the dataset is several gigabytes in size, this transfer process can take an excessive amount of time, even with a very good network connection.

On the cluster, with both V100 and A100 GPUs, the script initialized correctly. However, the training failed due to a memory issue, according to the error messages, despite being carried out under standard memory configurations. The appendix does not specify any memory threshold.

Disclaimer: This Reproducibility Report was crafted by volunteers with the goal of enhancing reproducibility in our research domain. The time period allocated for the reproducibility analysis was constrained by paper notification deadlines and camera-ready submission dates. Furthermore, the compute hours in the shared infrastructure (e.g., Chameleon Cloud) available to the authors of this report were limited and restricted the scope and quantity of experiments in the review phase. Consequently, the inability to reproduce certain artifacts within this evaluation should not be interpreted as definitive evidence of their irreproducibility. Limitations in the time allocated to this review and the compute resources available to the reviewers may have prevented a positive outcome. Furthermore, reviewers assess the reproducibility of the artifacts provided by the authors; however, they are not accountable for verifying that the artifacts support the main claims of the paper.